

Music Generation

Petra Galuscakova

petra.galuscakova@uis.no



Department of Electrical Engineering and Computer Science
University of Stavanger

March 10, 2026



Transformers for Music Generation

MuseGAN



- ▶ The model must be able to learn from and re-create the **sequential structure of music**
- ▶ Music generation presents additional challenges that are not present for text generation, namely generating pitch and rhythm
- ▶ Music is often **polyphonic**: there are several streams of notes played simultaneously on different instruments, which combine to create harmonies that are either dissonant (clashing) or consonant (harmonious)
- ▶ Unlike text data, music is a multipart, consists of many sounds that are not necessarily delivered at the same time, there is the interplay between different rhythms across the ensemble: e.g. a guitarist might play a flurry of quicker notes while the pianist holds a longer sustained chord.
- ▶ Generating music note by note is complex, because we often do not want all the instruments to change notes simultaneously.



- ▶ Many of the techniques for text generation can also be used for music generation



Transformers for Music Generation

MuseGAN

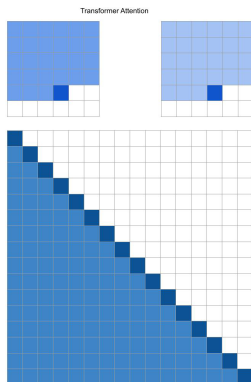


- ▶ The model we will be building here is a decoder Transformer, taking inspiration from OpenAI's MuseNet¹, which also utilizes a decoder Transformer trained to predict the next note given a sequence of previous notes
- ▶ In music generation tasks, the length of the sequence N grows large as the music progresses, and this means that the $N \times N$ attention matrix for each head becomes expensive to store and compute.
- ▶ We ideally do not want to clip the input sequence to a short number of tokens, as we would like the model to construct the piece around a long-term structure and repeat motifs and phrases from several minutes ago, as a human composer would.

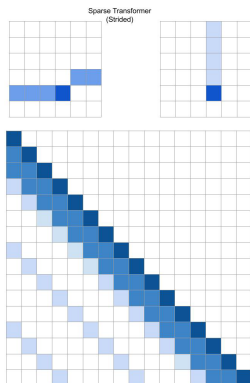
¹<https://openai.com/index/musenet/>



- ▶ To tackle this problem, **MuseNet** utilizes a form of Transformer known as a **Sparse Transformer**.
- ▶ Each output position in the attention matrix only computes weights for a subset of input positions, thereby reducing the computational complexity and memory required to train the model.
- ▶ MuseNet can therefore operate with full attention over 4,096 tokens and can learn long-term structure and melodic structure across a range of styles.

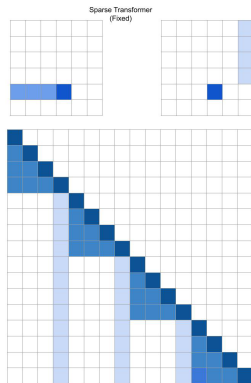


Strided Sparse Transformer



One of the approach of predicting the current pixel in two-dimensions is that the one head attends to l previous pixels and other head attends to every l th location where l is called stride.

Fixed Sparse Transformer



In the case where there is a lack of structure and periodicity in the data, the authors propose a method for two-dimensional factorization called fixed attention, where specific cells summarize the previous locations and propagate the information to all future cells.



MIDI Example



```
example_score.show()
```



The midi file starts with some metadata around the instrumentation, tempo, key, and time signature of the piece.

```
example_score.show("text")
```

```
{(0.0) <music21.metadata.Metadata object at 0xffff2b150b20>
(0.0) <music21.stream.Measure 1 offset=0.0>
(0.0) <music21.Instrument.Violoncello 'Solo Cello: Solo'
(0.0) <music21.Instrument.Violoncello 'Violoncello'>
(0.0) <music21.clef.BassClef>
(0.0) <music21.tempo.MetronomeMark Quarter=250.0>
(0.0) <music21.key.Key of G major>
(0.0) <music21.meter.TimeSignature 4/4>
(0.0) <music21.note.Rest 3.75ql>
(3.5) <music21.tempo.MetronomeMark Quarter=77.0>
(3.75) <music21.chord.Chord B3>
(4.0) <music21.stream.Measure 2 offset=4.0>
(0.0) <music21.chord.Chord G2 D3 B3>
(1.0) <music21.chord.Chord B3>
(1.25) <music21.chord.Chord A3>
(1.5) <music21.chord.Chord G3>
(1.75) <music21.chord.Chord F#3>
(2.0) <music21.chord.Chord G3>
(2.25) <music21.chord.Chord D3>
(2.5) <music21.chord.Chord E3>
(2.75) <music21.chord.Chord F#3>
(3.0) <music21.chord.Chord G3>
(3.25) <music21.chord.Chord A3>
(3.5) <music21.chord.Chord B3>
(3.75) <music21.chord.Chord C4>
```

This note starts on beat 4 of the piece (zero-indexed). It has duration of 1 beat (as the next note starts on beat 5) and consists of a chord of low G, D, and B.

This note starts on beat 6 of the piece. It has duration of a quarter of a beat (as the next note starts on beat 6.25) and consists of a single note – G.

This note starts a quarter of a beat before beat 8 of the piece. It has duration of a quarter of a beat and consists of a single note – high C.



- ▶ We start by looping over each score and extracting the **note** and **duration** of each element in the piece into **two separate text strings**, with elements separated by spaces.
- ▶ We encode the key and time signature of the piece as special symbols, with zero duration.

Notes string

```
START G:major 4/4TS rest B3 B3 B3 A3 G3 F#3 G3 D3 E3 F#3 G  
3 A3 B3 C4 D4 B3 G3 F#3 G3 E3 D3 C3 B2 C3 D3 E3 F#3 G3 A3 B  
3 C4 A3 G3 F#3 G3 E3 F#3 G3 A2 D3 F#3 G3 A3 B3 C4 A3 B3 ...
```

Duration string

```
0.0 0.0 0.0 3.75 0.25 1.0 0.25 0.25 0.25 0.25 0.25 0.25 0.  
25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25  
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25  
0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25 0.25  
0.25 0.25 0.25 ...
```

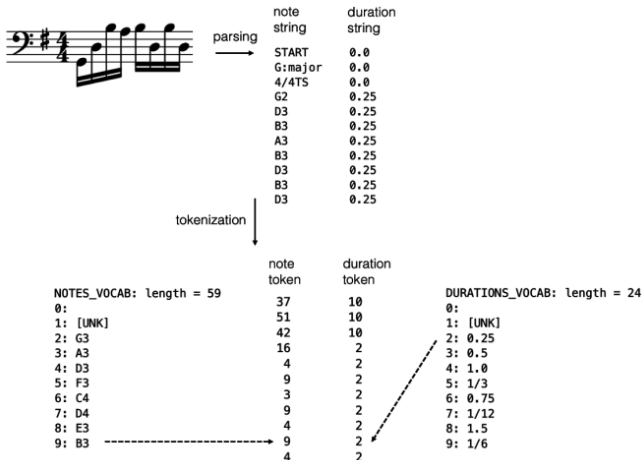


- ▶ A key difference between music and text generation is that we need to build a model that can handle the note and duration prediction simultaneously—i.e., there are two streams of information that we need to handle.
- ▶ The words are the note–duration combinations, and we should try to build a model that predicts the next note and duration, given a sequence of previous notes and durations.



- ▶ To create the dataset that will train the model, we first need to tokenize each note and duration, exactly as we did previously for each word in a text corpus.
- ▶ We can achieve this by using a TextVectorization layer, applied to the notes and durations separately.

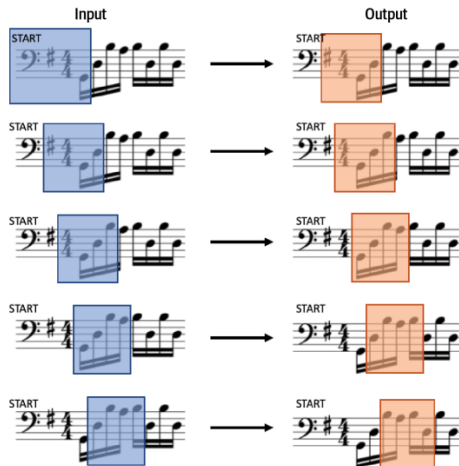
Tokenization (cont.)





- ▶ The final step of preprocessing is to create the training set that we will feed to our Transformer.
- ▶ We do this by splitting both the note and duration strings into chunks of 50 elements, using a sliding window technique.
- ▶ The output is simply the input window shifted by one note, so that the Transformer is trained to predict the note and duration of the element one timestep into the future, given previous elements in the window.

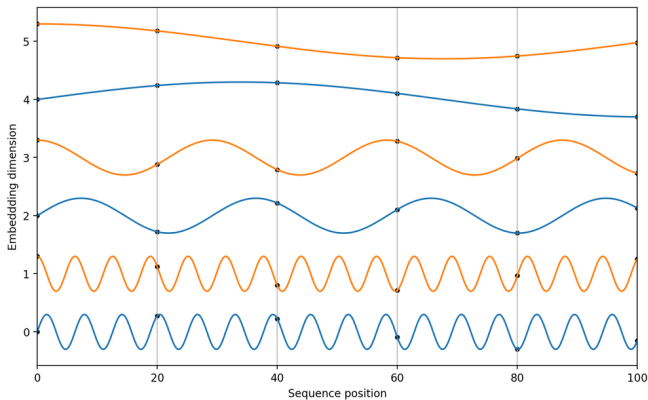
Creating the Training Set (cont.)



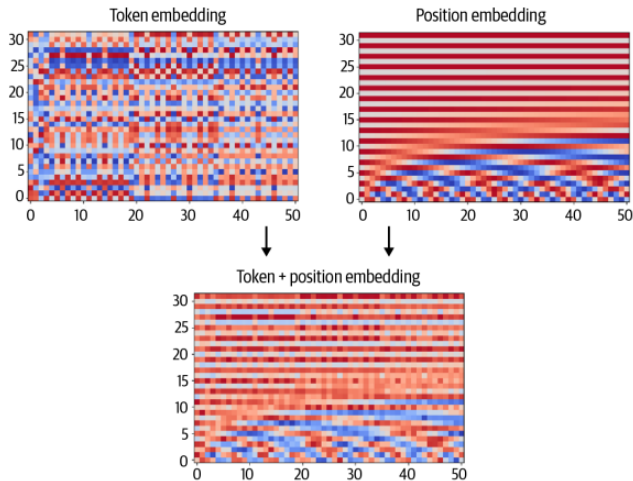


- ▶ To encode the position of each token in a simple embedding layer, we need to define a maximum length (N) that the sequence could be
- ▶ It is then impossible to extrapolate to sequences that are longer than this maximum length
- ▶ To circumvent this problem, we can switch to using a sine position embedding.
- ▶ Specifically, the following function is used to convert the position of the word (pos) in the input sequence into a unique vector of length d:
- ▶ $PE_{pos,2i} = \sin\left(\frac{pos}{10,000^{2i/d}}\right)$
- ▶ $PE_{pos,2i+1} = \cos\left(\frac{pos}{10,000^{(2i+1)/d}}\right)$
- ▶ Each position thus has its own unique encoding, which is a specific combination of the different wavelengths.

Sine Position Encoding (cont.)



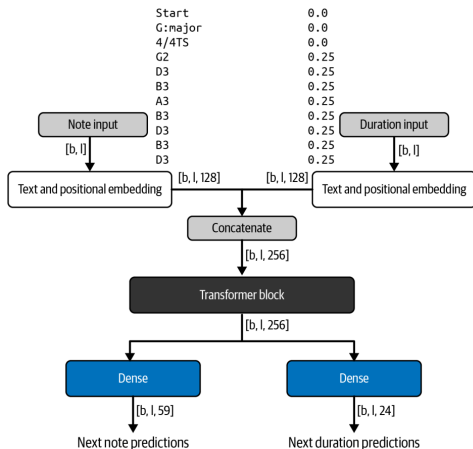
Sine Position Encoding (cont.)





- ▶ We now have two input streams (notes and durations) and two output streams (predicted notes and durations). We therefore need to adapt the architecture of our Transformer to cater for this.
- ▶ Many ways of handling the dual stream of inputs:
 - Create tokens that represent each note–duration pair and then treat the sequence as a single stream of tokens: this cannot represent note–duration pairs that have not been seen in the training set.
 - Instead, we choose to embed the note and duration tokens separately and then use a concatenation layer to create a single representation of the input that can be used by the downstream Transformer. Similarly, the output from the Transformer block is passed to two separate dense layers, which represent the predicted note and duration probabilities.

Multiple Inputs and Outputs





- ▶ We can generate a musical passage using the iterative technique for generating text sequences:
 - 1. Given the current sequence (of notes and durations), the model predicts two distributions, one for the next note and one for the next duration.
 - 2. We sample from both of these distributions, using a temperature parameter to control how much variation we would like in the sampling process.
 - 3. The chosen note and duration are appended to the respective input sequences.
 - 4. The process repeats with the new input sequences for as many elements as we wish to generate.

Examples of Generated Music



Epoch 2

The musical score for Epoch 2 is displayed on two staves. The top staff is in treble clef with a key signature of one sharp (F#) and a 4/4 time signature. It contains a melody of eighth and sixteenth notes. The bottom staff is in bass clef with a key signature of one sharp (F#) and a 4/4 time signature. It contains a bass line of eighth and sixteenth notes, ending with a double bar line.

Epoch 10

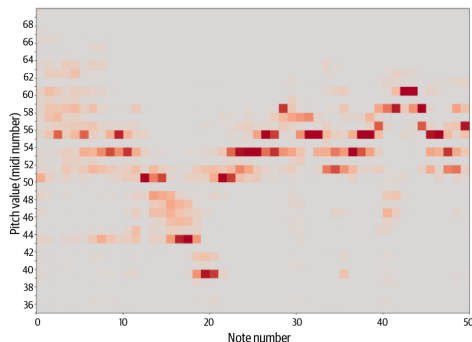
The musical score for Epoch 10 is displayed on two staves. The top staff is in treble clef with a key signature of one sharp (F#) and a 4/4 time signature. It contains a melody of eighth and sixteenth notes. The bottom staff is in bass clef with a key signature of one sharp (F#) and a 4/4 time signature. It contains a bass line of eighth and sixteenth notes, ending with a double bar line.

Epoch 20

The musical score for Epoch 20 is displayed on two staves. The top staff is in treble clef with a key signature of one sharp (F#) and a 4/4 time signature. It contains a melody of eighth and sixteenth notes. The bottom staff is in bass clef with a key signature of one sharp (F#) and a 4/4 time signature. It contains a bass line of eighth and sixteenth notes, ending with a double bar line.



- ▶ The music is becoming more sophisticated as training progresses.
- ▶ To begin with, the model plays it safe by sticking to the same group of notes and rhythms.
- ▶ By epoch 10, the model has begun to generate small runs of notes, and by epoch 20 it is producing interesting rhythms and is firmly established in a set key.



The distribution of possible next notes over time (at epoch 20): the darker the square, the more certain the model is that the next note is at this pitch.

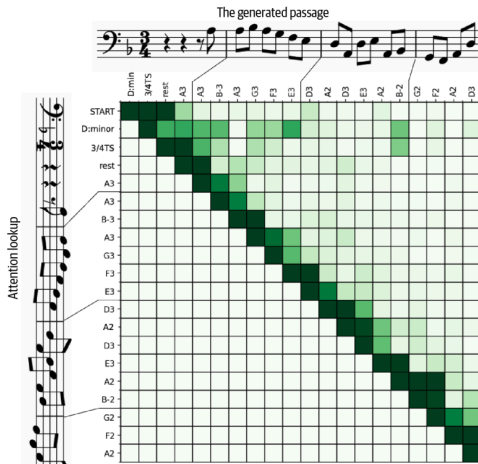


- ▶ The model has clearly learned which notes belong to particular keys, as there are gaps in the distribution at notes that do not belong to the key.
- ▶ For example, there is a gray gap along the row for note 54 (corresponding to $G\flat/F\sharp$). This note is highly unlikely to appear in a piece of music in the key of $E\flat$ major.
- ▶ The model establishes the key early on in the generation process, and as the piece progresses, the model chooses notes that are more likely to feature in that key by attending to the token that represents it.



- ▶ The model has learned Bach's characteristic style of dropping to a low note on the cello to end a phrase and bouncing back up again to start the next (see note 20).

Observations of Generated Music cont.



The color of each square in the matrix indicates the amount of attention given to each position on the vertical axis, at the point of predicting the note on the horizontal axis.



- ▶ For the initial key signature, time signature, and rest, the network chose to place almost all of its attention on the START token, as these artifacts always appear at the start of a piece of music—once the notes start flowing the START token essentially stops being attended to.
- ▶ We can see that the network places most attention on approximately the last two to four notes and rarely places significant weight on notes more than four notes ago.
- ▶ This is due as there is probably enough information contained in the previous four notes to understand how the phrase might continue.
- ▶ Additionally, some notes attend more strongly back to the key signature of D minor—for example, the E3 (7th note of the piece) and B-2 (Bb—14th note of the piece).
- ▶ The network must look back at the key signature in order to tell that there is a Bb in the key signature (rather than a B natural)



- ▶ The penultimate note (A2) is not particularly attentive to the B-2 three notes back, but is slightly more attentive to the A2 four notes back. It is more interesting for the model to look at the A2 that falls on the beat, rather than the B-2 off the beat, which is just a passing note.



- ▶ The challenge lies in how to represent the different lines of music as a single sequence of tokens.
- ▶ In the previous section we decided to split the notes and durations of the notes into two distinct inputs and outputs of the network, but we also saw that we could have interleaved these tokens into a single stream.
- ▶ We can use the same idea to handle polyphonic music. Two different approaches will be introduced here: grid tokenization and event-based tokenization².

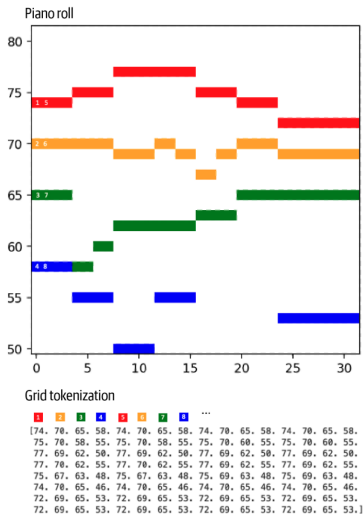
²<https://arxiv.org/abs/1809.04281>



There are four distinct parts (soprano [S], alto [A], tenor [T], bass [B]), written on different staves.



- ▶ We can imagine drawing this music on a grid, where the y-axis represents the pitch of the note and the x-axis represents the number of 16th-notes
- ▶ If the grid square is filled, then there is a note playing at that point in time.
- ▶ All four parts are drawn on the same grid. This grid is known as a piano roll
- ▶ We can serialize the grid into a stream of tokens by moving first through the four voices, then along the timesteps in sequence. This produces a sequence of tokens $S_1, A_1, T_1, B_1, S_2, A_2, T_2, B_2, \dots$, where the subscript denotes the timestep.



<https://musiclab.chromeexperiments.com/Piano-Roll/>



- ▶ We would then train our Transformer on this sequence of tokens, to predict the next token given the previous tokens.
- ▶ We can decode the generated sequence back into a grid structure by rolling the sequence back out over time in groups of four notes (one for each voice).
- ▶ This technique works surprisingly well, despite the same note often being split across multiple tokens with tokens from other voices in between.
- ▶ Disadvantages:
 - There is no way for the model to tell the difference between one long note and two shorter adjacent notes of the same pitch. This is because the tokenization does not explicitly encode the duration of notes, only whether a note is present at each timestep.
 - This method requires the music to have a regular beat that is divisible into reasonably sized chunks.



- It is not obvious how we might add other components, such as dynamics (how loud the music is) or tempo changes.



- ▶ A more flexible approach is to use event-based tokenization.
- ▶ This can be thought of as a vocabulary that literally describes how the music is created as a sequence of events, using a rich set of tokens.
- ▶ We use three types of tokens:
 - NOTE_ON⟨pitch⟩ (start playing a note of a given pitch)
 - NOTE_OFF⟨pitch⟩ (stop playing a note of a given pitch)
 - TIME_SHIFT⟨pitch⟩ (shift forward in time by a given step)
- ▶ This vocabulary can be used to create a sequence that describes the construction of the music as a set of instructions.



Event tokenization

```
[NOTE_ON<74>, NOTE_ON<70>, NOTE_ON<65>, NOTE_ON<58>  
, TIME_SHIFT<1.0>  
, NOTE_OFF<74>, NOTE_OFF<65>, NOTE_OFF<58>  
, NOTE_ON<75>, NOTE_ON<58>, NOTE_ON<55>  
, TIME_SHIFT<0.5>  
, NOTE_OFF<58>, NOTE_ON<60>  
, TIME_SHIFT<0.5>  
, NOTE_OFF<75>, NOTE_OFF<70>, NOTE_OFF<60>, NOTE_OFF<55>  
, NOTE_ON<77>, NOTE_ON<69>, NOTE_ON<62>, NOTE_ON<50>  
, TIME_SHIFT<1.0>  
, NOTE_OFF<69>, NOTE_OFF<50>, NOTE_ON<70>, NOTE_ON<55>  
, TIME_SHIFT<0.5>  
, NOTE_OFF<70>, NOTE_ON<69>  
, TIME_SHIFT<0.5>  
, NOTE_OFF<77>, NOTE_OFF<69>, NOTE_OFF<62>, NOTE_OFF<55>  
, ...]
```




- ▶ A more flexible approach is to use event-based tokenization.
- ▶ We could easily incorporate other types of tokens into this vocabulary, to represent dynamic and tempo changes for subsequent notes.
- ▶ This method also provides a way to generate triplets with `TIME_SHIFT<0.33>` tokens.
- ▶ Overall, it is a more expressive framework for tokenization, though it is also potentially more complex for the Transformer to learn inherent patterns in the training set music, as it is by definition less structured than the grid method.



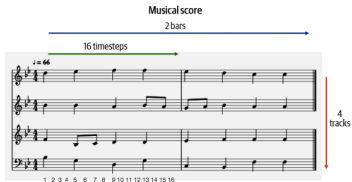
Transformers for Music Generation

MuseGAN

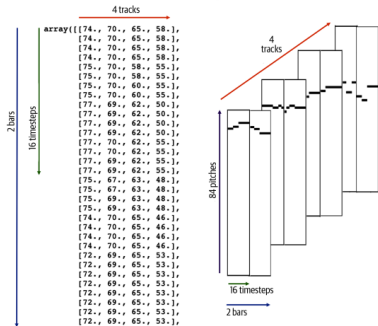


- ▶ We can treat this piano roll as a picture and utilize image generation methods instead of sequence generation techniques
- ▶ Instead of using Transformers, we can apply GAN

GAN Data Preparation

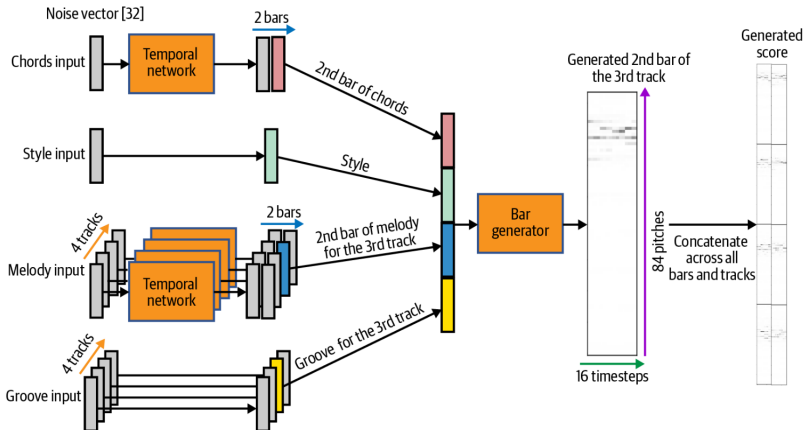


Raw data → Transformed data





- ▶ Like all GANs, MuseGAN consists of a generator and a critic.
- ▶ The generator tries to fool the critic with its musical creations, and the critic tries to prevent this from happening by ensuring it is able to tell the difference between the generator's forged Bach chorales and the real thing.
- ▶ Where MuseGAN differs is in the fact that the generator doesn't just accept a single noise vector as input, but instead has four separate inputs, which correspond to four different characteristics of the music: chords, style, melody, and groove.
- ▶ By manipulating each of these inputs independently we can change high-level properties of the generated music.





- ▶ Chords and melody inputs are first passed through a temporal network that outputs a tensor with one of the dimensions equal to the number of bars to be generated.
- ▶ The style and groove inputs are not stretched temporally in this way, as they remain constant through the piece.
- ▶ Then, to generate a particular bar for a particular track, the relevant outputs from the chords, style, melody, and groove parts of the network are concatenated to form a longer vector.
- ▶ This is then passed to a bar generator, which ultimately outputs the specified bar for the specified track.
- ▶ By concatenating the generated bars for all tracks, we create a score that can be compared with real scores by the critic.



<https://hermandong.com/musegan/results.html>



- ▶ The job of a temporal network—a neural network consisting of convolutional transpose layers—is to transform a single input noise vector of length $Z_DIM = 32$ into a different noise vector for every bar



► Chords:

- The chords input is a single noise vector of length Z_DIM .
- This vector's job is to control the general progression of the music over time, shared across tracks, so we use a TemporalNetwork to transform this single vector into a different latent vector for every bar.
- While we call this input chords, it really could control anything about the music that changes per bar, such as general rhythmic style, without being specific to any particular track.

► Style:

- The style input is also a vector of length Z_DIM .
- This is carried forward without transformation, so it is the same across all bars and tracks.
- It can be thought of as the vector that controls the overall style of the piece (i.e., it affects all bars and tracks consistently).



► Melody:

- The melody input is an array of shape $[N_TRACKS, Z_DIM]$ - that is, we provide the model with a random noise vector of length Z_DIM for each track.
- Each of these vectors is passed through a track-specific TemporalNetwork, where the weights are not shared between tracks.
- The output is a vector of length Z_DIM for every bar of every track.
- The model can therefore use these input vectors to fine-tune the content of every single bar and track independently.

► Groove:

- The groove input is also an array of shape $[N_TRACKS, Z_DIM]$ - a random noise vector of length Z_DIM for each track.
- Unlike the melody input, these vectors are not passed through the temporal network but instead are fed straight through, just like the style vector.



- Therefore, each groove vector will affect the overall properties of a track, across all bars.



- ▶ The bar generator receives four latent vectors—one from each of the chord, style, melody, and groove components.
- ▶ These are concatenated to produce a vector of length $4 * Z_DIM$ as input.
- ▶ The output is a piano roll representation of a single bar for a single track
- ▶ The bar generator is just a neural network that uses convolutional transpose layers to expand the time and pitch dimensions of the input vector.
- ▶ We create one bar generator for every track, and weights are not shared between tracks.

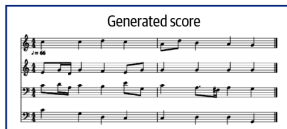


- ▶ The critic tries to distinguish full multitrack, multibar scores created by the generator from real music excerpts.
- ▶ It is a convolutional neural network, consisting mostly of Conv3D layers that collapse the score into a single output prediction.
- ▶ The output from the generator is an array of values in the range $[-1, 1]$.
- ▶ To convert this to a single note for each track, we choose the note with the maximum value over all 84 pitches for each timestep.

Analysis of the MuseGAN



Generated score



Changing the style noise



Closest real score



Changing the melody noise
for the top line only



Changing the chord noise



Changing the groove noise
for the bottom line only

